IOTA JU Redesign
Sourasish: Platform and Release Integration

Implementation handout

21 October 2026

1 Your job

You own the application foundation, shared platform configuration, public content reads, preview, cache refresh, legacy URL routing, deployment and operational handover. Build these pieces so Ritendu can safely model content, Pritish can render it, and Shreyam can import it.

The app itself is not scaffolded yet. This repository already contains planning files. Keep those files and the existing Git history. Do not run a scaffold command over the non-empty repository. The public site, Payload admin and APIs are one Next.js application on Vercel. PostgreSQL and file storage are separate providers.

Plain language: Payload is the editor and content system inside the site. A content reader is the server-side function that gives a page only the public, published fields it needs. Preview is a private view of a draft. Cache refresh makes a published change appear on the public site.

2 Install and configure

Install Git, a supported Node.js LTS version (at least 20.18.1 under the current Payload requirement), and pnpm. The exact Node and package versions are selected at kickoff and written in `.nvmrc`, `package.json` and `pnpm-lock.yaml`. Use that same lockfile on every workstation. Do not install a second package manager into the project.

Bootstrap into a temporary, empty folder because this repository already has files:

```
pnpm create next-app@latest /tmp/iota-ju-next-scaffold --yes
```

Review the generated files, then copy only the application scaffold/configuration into the repository root. Preserve `.git`, `README.md`, `main.tex` and `docs/`; merge the generated ignore rules instead of replacing the repository's rules. Do not keep a nested app directory or nested package.json.

Install the compatible Payload/Postgres packages after checking the official setup guide. Use the versions allowed by the Next.js compatibility table for the selected Payload release:

```
pnpm add payload @payloadcms/next @payloadcms/db-postgres
pnpm add @payloadcms/richtext-lexical sharp
pnpm add -D typescript eslint @types/node @types/react @types/react-dom
pnpm add -D vitest @playwright/test
```

If a generated Payload template already installed a package, keep one copy. Do not install GraphQL unless the team chooses to use it. Use Payload's version-compatible route template rather than manually recreating its admin/API route files.

3 Files you own

Files or folders	What they do
`package.json`, `pnpm-lock.yaml`, `.nvmrc`	Pinned packages, scripts, package manager and Node version. You are the only routine editor of these shared root files.
`next.config.mjs`, `tsconfig.json`, `eslint.config.mjs`	Next/Payload plugin, TypeScript aliases and lint rules. Keep module format compatible with Payload's plugin.
`payload.config.ts`	Create the initial config and Payload route wiring, then transfer config/types/migration ownership to Ritendu after P01.
`src/app/(payload)/`	Version-matched Payload admin, REST and optional GraphQL route files from the official blank template.
`src/env.ts`, `.env.example`	Validate required environment values; show names and safe placeholders only.
`src/lib/content/`	Small server-side functions for published project/event/person/global reads.
`src/lib/preview.ts`, `src/lib/content/invalidation.ts`	Authorized draft preview and cache refresh helpers.
`src/app/api/preview/`, `src/app/api/exit-preview/`	Preview entry/exit handlers, with safe redirect validation.
`src/lib/redirects/`	Resolve inventoried legacy paths without loops or external redirects.
`.github/workflows/quality.yml`, `docs/environments.md`, `docs/operations.md`	CI, environment setup and recovery instructions.
`tests/public-content.test.ts`, `tests/e2e/`	Focused reader, preview, publishing, legacy-route and operational checks.

4 P01: create the reproducible application base

Steps

1. Create the Next.js App Router and TypeScript scaffold in a temporary folder, then bring only its source/configuration into this repository.
2. Add Payload and its PostgreSQL adapter, Lexical editor, and image processing support. Wrap the Next config with Payload's official Next plugin.
3. Add the Payload route group under `src/app/(payload)/`. Keep public routes in the separate `src/app/(site)/` group owned by Pritish.
4. Add the scripts `dev`, `build`, `start`, `lint`, `typecheck`, `test`, `test:e2e`, `payload` and `db:migrate`. The Payload CLI script must point to `src/payload.config.ts`; `db:migrate` must run the CLI migration

command against the explicitly selected environment.

5. Add a safe `.env.example` containing `DATABASE_URL`, `PAYLOAD_SECRET` and `SERVER_URL`, each with a local placeholder. Add provider-specific storage names only after provider selection. Validate required variables on the server; never put a secret in a `NEXT_PUBLIC_` name.
6. Record the tested Node, Next.js, Payload, TypeScript and pnpm versions in `docs/environments.md`. Commit the lockfile.

P01 files: root package/config files above; `src/app/(payload)/`; `src/env.ts`; `.env.example`; `vitest.config.ts`; `playwright.config.ts`; one environment-validation test.

Check:

```
pnpm install --frozen-lockfile
pnpm typecheck
pnpm lint
pnpm test
pnpm build
```

A second person must clone the repository and run the same commands without files from the old site.

5 P02: protected staging and CI

Wait for the orchestrator to approve provider ownership and budget. Configure a club-owned Vercel project with staging/preview and production environments. Use separate database and storage scopes; do not copy production records into public PR previews. Keep secrets unavailable to untrusted fork builds.

Create `.github/workflows/quality.yml` to run install, lint, typecheck, tests and build. Document environment names and setup in `docs/environments.md`. Run migrations as an explicit, serialized action against the intended database. Never use automatic schema synchronization that can drop or rewrite production data.

Check: one successful PR preview; a migration against an empty staging database; prove staging credentials cannot write to production. Do not print secrets in logs.

6 P03: public content readers

Coordinate the field contract with Ritendu first. Start with projects and add other readers only as their CMS collections arrive. Keep each function small and server-only; do not make pages call the same Next application over HTTP.

Create: `src/lib/content/projects.ts`, `src/lib/content/events.ts`, `src/lib/content/people.ts`, `src/lib/content/globals.ts`, with `tests/public-content.test.ts`. Return only fields Pritish's page needs. Apply published-only filtering and explicit access control even when using Payload's local API. Bound relationship depth and never return user records, internal legacy keys or private master-media URLs.

Check: anonymous requests cannot obtain a draft by passing draft/filter options; test malformed slugs, empty data, missing/unpublished detail, database failure and public field projection. Missing content becomes a 404; infrastructure errors remain diagnosable as errors.

7 P04: private draft preview

After Ritendu's user roles and Pritish's first project template exist, create preview entry/exit handlers and `src/lib/preview.ts`. Validate the session, preview token, record and relative destination on the server. Reject arbitrary external return URLs. Use private/no-store/noindex response behavior. Ensure the public version remains unchanged while a draft is previewed.

Coordinate: Ritendu wires CMS preview links; Shreyam secures private draft assets; Pritish uses the same public template for preview and live pages.

Check: `pnpmtest:e2e--tests/e2e/preview.spec.ts`. Try signed out, expired session, valid editor, external redirect, exit preview and direct private media URL.

8 P05: publish refresh and revoke

Agree with Ritendu on a small event payload that includes collection, record ID, old/new slug and affected public relationships. Refresh only after the write succeeds. Account for transaction timing and observable failure/retry behavior.

Create these files:

- `src/lib/content/invalidation.ts`
- `docs/cache-dependencies.md`
- `tests/e2e/publishing.spec.ts`

The first slice covers one project detail, project list and home teaser. Later add event, team, alumni, global and related-media surfaces as the records arrive.

Check: test publish, slug change, draft save, failed refresh and unpublish on staging. Record timings. Public updates appear within 60 seconds. Revoked private items disappear immediately.

9 P06: legacy URLs and not-found behavior

Wait for Shreyam's route inventory and Ritendu's redirect records. Build the smallest resolver that handles exact old paths, encoded spaces, case and singular `/project/` routes. Preserve Pritish's ownership of page appearance and metadata.

Create: `src/lib/redirects/resolve.ts`, `tests/e2e/legacy-routes.spec.ts`; add cycle/invalid destination checks to the build or test. Unknown pages return a real 404; database failures do not pretend the page is missing. Old Q&A URLs get only an orchestrator-approved retirement response.

10 P07: operations and recovery

Create a restricted readiness check and `docs/operations.md`. Coordinate with Ritendu on database backup/migrations and Shreyam on originals and delivery media. Configure useful failure monitoring without logging secrets or private content.

Check: restore both database and media into isolated staging, verify record relationships and file counts, record elapsed recovery time, and document account ownership and budget alerts. A Git rollback is not a database restore.

11 P08: production release

Do not switch the public domain or perform the final production import until the orchestrator explicitly approves the release gate. Coordinate the final manifest/import with Shreyam, content/users with Ritendu, and visual/browser checks with Pritish.

Create: `docs/releases/launch.md` with release commit, final import report, approvals, checks, domain steps and a tested rollback. Keep the old site available. Do not bundle legacy deletion into launch.

12 Your handoff boundaries

After P01, transfer `payload.config.ts`, generated Payload types and migration ordering to Ritendu. Keep root dependencies, lockfile, TypeScript/lint/test configuration, CI and public reader/cache/preview/redirect runtime under your ownership. Coordinate Media route/config additions with Shreyam and Ritendu; do not overwrite their uncommitted work.

13 Official setup references

The current official Next.js installation guide documents the framework setup. The official Payload installation guide currently requires Node.js 20.18.1 or newer and lists compatible Next ranges, package installation and route files. Use the PostgreSQL adapter, not MongoDB. Current Vercel Function limits limit request and response bodies to 4.5 MB, so large media must go directly to approved storage. Checked 21 October 2026.